

Homework 5

For this homework you will work with your group on a **GitHub** repo that creates a nice website to answer the questions below (you can reuse a previously created repo if you'd like! Just turn in the link to the correct webpage). One group member needs to turn in the link to this website!

Assignment Tasks

A `.qmd` file that creates a final website is the final product. I would recommend each group member create their own `.qmd` file for their version of the solutions. Then have the group decide on how to merge the solutions for the final file. You don't have to follow this structure though!

- In the final document, answers to the questions below along with a discussion of what you are doing (via markdown) and comments in the code should be included.

Task 1: Conceptual Questions

Create a markdown list with the following questions:

1. What is the purpose of the `lapply()` function? What is the equivalent `purrr` function?
2. Suppose we have a list called `my_list`. Each element of the list is a numeric data frame (all columns are numeric). We want use `lapply()` to run the code `cor(numeric_matrix, method = "kendall")` on each element of the list. Write code to do this below! (I'm really trying to ask you how you specify `method = "kendall"` when calling `lapply()`)
3. What are two advantages of using `purrr` functions instead of the `BaseR` apply family?
4. What is a side-effect function?
5. Why can you name a variable `sd` in a function and not cause any issues with the `sd` function?

Task 2 - Writing R Functions

1. When we start doing machine learning later in the course, a common metric used to evaluate predictions is called Root Mean Square Error (RMSE).

For a given set of responses, $y_1, \dots, y_n$ (variable of interest that we want to predict) and a set of corresponding predictions for those observations, $\hat{y}_1, \dots, \hat{y}_n$ the RMSE is defined as

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Write a basic function (call it `getRMSE()`) that takes in a *vector* of responses and a *vector* of predictions and outputs the RMSE.

- If a value is missing for the vector of responses (i.e. an NA is present), allow for additional arguments to the `mean()` function (elipses) that removes the NA values in the computation.
2. Run the following code to create some response values and predictions.

```
set.seed(10)
n <- 100
x <- runif(n)
resp <- 3 + 10 * x + rnorm(n)
pred <- predict(lm(resp ~ x), data.frame(x))
```

- Test your RMSE function using this data.
 - Repeat after replacing two of the response values with missing values (`NA_real_`).
 - Test your RMSE function with and without specifying the behavior to deal with missing values.
3. Another common metric for evaluating predictions is mean absolute deviation given by

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Write a function called `getMAE()` that follows the specifications of the `getRMSE()` function.

4. Run the following code to create some response values and predictions.

```
set.seed(10)
n <- 100
x <- runif(n)
resp <- 3 + 10 * x + rnorm(n)
pred <- predict(lm(resp ~ x), data.frame(x))
```

- Test your MAE function using this data.
 - Repeat after replacing two of the response values with missing values (`NA_real_`).
 - Test your MAE function with and without specifying the behavior to deal with missing values.
5. Let's create a **wrapper** function that can be used to get either or both metrics returned with a single function call. Do not rewrite your above two functions, call them inside the wrapper function (we would call the `getRMSE()` and `getMAE()` functions **helper** functions). When returning your values, give them appropriate names.
- The function should check that two numeric (atomic) vectors have been passed (consider `is.vector()`, `is.atomic()`, and `is.numeric()`). If not, a message should print and the function should exit.
 - The function should return both metrics by default and include names. The behavior should be able to be changed using a character string of metrics to find.
6. Run the following code to create some response values and predictions.

```
set.seed(10)
n <- 100
x <- runif(n)
resp <- 3 + 10 * x + rnorm(n)
pred <- predict(lm(resp ~ x), data.frame(x))
```

- Test your new function using this data. Call it once asking for each metric individually and once specifying both metrics
- Repeat with replacing two of the response values with missing values (`NA_real_`).
- Finally, test your function by passing it incorrect data (i.e. a data frame or something else instead of vectors)

To Turn In

One member should submit the link to the nicely rendered site on Moodle!